

Section 1.3

Basic C++ Classes

1. Principles
2. Code organization
3. Default arguments
4. Default constructors
5. Destructors
6. Object duplication
7. Copy constructors
8. Conversion constructors

1.3.1 Principles

- A *class definition* includes:
 - a class name
 - the class members
 - data members
 - member functions
- Access specifiers are required for all class members
 - public, protected, private
- Default access specifier is *private*

Access Specifiers

- Industrial grade software is typically huge
 - potentially millions of lines of code
 - possibly hundreds of developers over many years
 - you must protect your runtime objects from bad code
- The philosophy:
 - to protect the content of your classes, you must restrict access
 - principle of least privilege
 - make all your class members as private as possible
 - ... more on this later...

Access Specifiers (cont.)

- A class definition specifies access level for:
 - every data member
 - every member function
- Access levels:
 - public
 - protected
 - private

Access Specifiers (cont.)

- Public access
 - the class member is visible by all objects and all global functions
- Protected access
 - the class member is visible by objects of sub-class types only
 - this access level only makes sense when using inheritance
- Private access
 - the class member is not visible to objects of other class types, or to global functions
 - other objects of the same class can access private members

Member Function Implementation

- What is a *function implementation*?
 - it's the sequence of statements for a function
 - it's the "body" of the function
- In a correctly organized C++ program:
 - the class definition is stored in one file
 - the member function implementations are stored in another file
- **coding example <p1>**

1.3.2 Code Organization

- For very small programs
 - the function implementations can be inside the class definition
 - this is the only way to define classes in Java
 - it gets very messy, very quickly
- For all other programs (and all course work)
 - the function implementations must be in a **separate** file
 - the function prototypes must still be declared in the class definition

Code Organization (cont.)

- Each class is defined using **two files**:
 - a *header* file (using the `.h` file extension)
 - this contains the *class definition*
 - the class definition contains:
 - the data member declarations
 - the member function *prototypes* (**not the code!**)
 - a *source* file (using the `.cc` file extension)
 - this contains the *member function implementations* (the actual code)
 - static data member initializations
 - ... more on this later ...
- **coding example <p2>**

Best Practices for Code Packaging

- Basic principles
 - class users
 - these are the developers who use your class in their code
 - remember
 - very few professional developers code directly for the end users
 - you must learn to write code for *other developers*
 - what your class users need to know
 - the class name
 - its public members
 - data members and member functions
 - sometimes its protected members too

Best Practices for Code Packaging (cont.)

- What is a *class interface*?
 - it is **not** a Java interface!
 - Java uses the word *interface* in a very specific, non-standard way
 - it's not the general computing meaning of the word *interface*
 - in OO design, a *class interface* is what your class users need to know to use your class:
 - the class name
 - the class's public members

Best Practices for Code Packaging (cont.)

- Your class users need:
 - your class definition
 - this is contained inside the class header file
 - the header file must be included in the class user's code
 - using the C++ `#include` preprocessor directive
 - your class object code
 - this contains the class source code, after it is compiled
 - the class users need the object code to *link* into their program
- Your class users do **not** need:
 - your class source code

Class Header File

- The header file contains the *class definition*:
 - the data members
 - the member function prototypes
- Use include guards!
 - this protects against multiple `includes` and class re-definitions
- Do not confuse the word “file” with its contents!
 - a **file** can contain any type of information
 - my program for Assignment #1
 - my curriculum vitae (CV), my recipe for chocolate chip cookies
 - a **class definition** defines a C++ class and its members
 - we just happen to store the class definition inside a header file

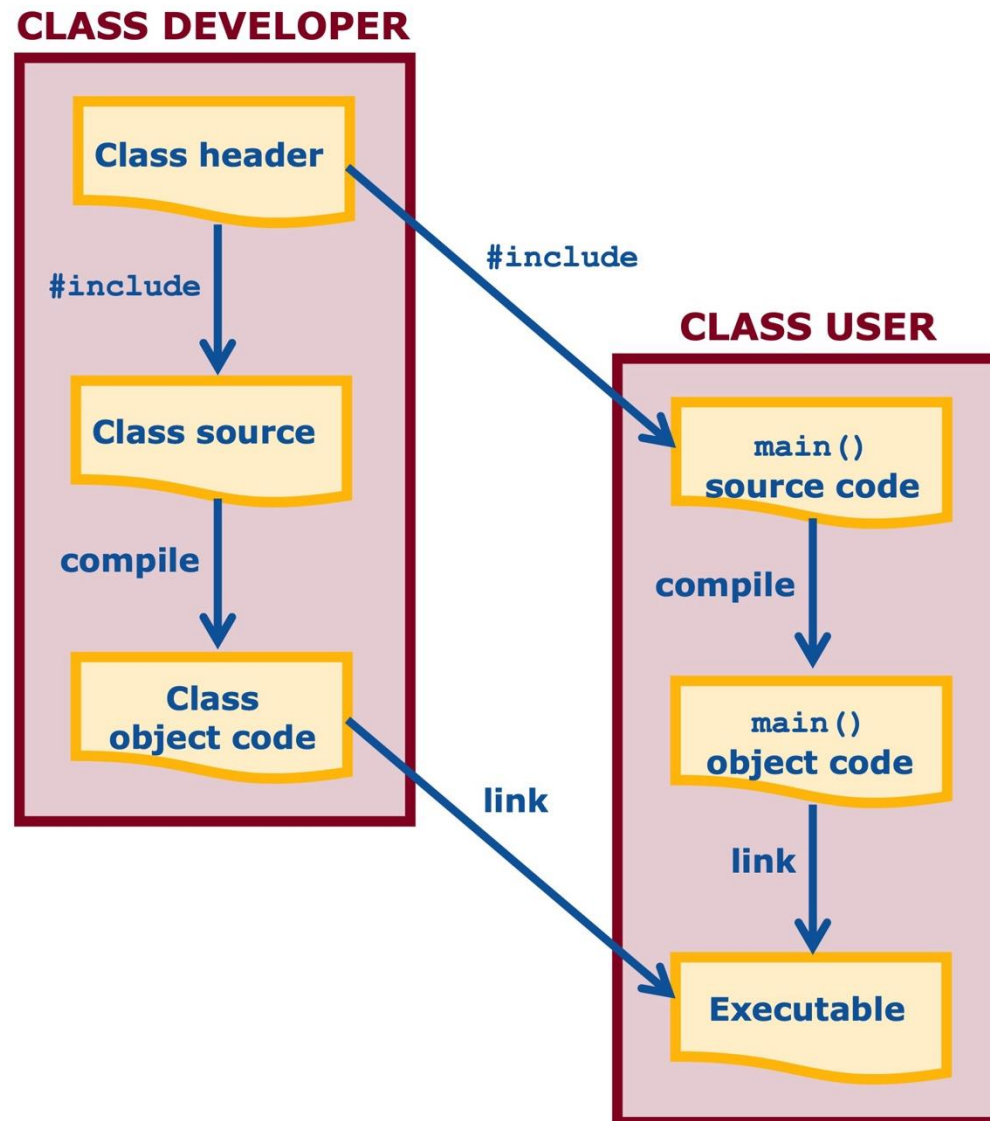
Class Header File (cont.)

- **NEVER** `#include` source code !!!
 - you would be forcing re-compilation of all class users' code
 - compilation is slow and error-prone
 - understand what belongs in the header file vs. the source file
 - you must:
 - #1- compile each source file separately into object code
 - #2- link all the object code into one executable
 - class users will have to re-link their code to yours, which is faster

Class Source File

- The source file contains all class-related source code:
 - all member function implementations
 - warning: by default, all functions are **global**
 - use the *scope resolution operator* to resolve a function to its class
 - the static data member initializations
 - ... more on this later...

Program Building with Class Files



- Think:

- if a class implementation changes, what needs to be recompiled?

1.3.3 Default Arguments

- Terminology
 - a function *argument* is the same as a parameter value
 - a *default argument* is a predefined default value for a parameter, if no parameter value is sent by the calling function
- From a calling function's point of view:
 - it sends arguments to the called function
 - it may omit arguments in the function call, if default values are defined
- From a called function's point of view:
 - it receives parameter values from the calling function
 - some parameters could have default values

Default Arguments (cont.)

- Characteristics of default arguments:
 - they are specified in the function prototype
 - one or more parameters may have default values
 - defaulted parameters must be right-most in the parameter list
 - they can be used in global functions or member functions
- Common use of default arguments
 - for combining a default constructor with a multi-parameter one
 - but only **one** default constructor is allowed for each class
- **coding example** <p3>

1.3.4 Default Constructors

- What is a *default constructor*?
 - it's a constructor that takes *no parameters*
 - it's a constructor where all parameters have default arguments
- Characteristics of a default constructor:
 - it's a member function of a class
 - only **one** default constructor can exist for each class
 - we can still define other constructors that do take parameters, but that's unusual
 - an empty default constructor is provided automatically
 - if no constructors are defined
 - it may be called implicitly or explicitly

Default Constructors (cont.)

- Uses of default constructors:
 - they initialize the data members of an object with default values
 - a good constructor must initialize all the data members
- Default constructors are called *implicitly*:
 - when an object is declared, with no parameters specified
 - when an array of objects is declared
 - the default constructor is automatically called for each element
 - when memory for an object is dynamically allocated
 - using the `new` operator
 - ... more on this later...
- **coding example <p4>**

1.3.5 Destructors

- Characteristics of a *destructor*:
 - it's a member function of a class
 - only **one** destructor can exist for each class
 - it takes no parameters
 - an empty destructor is provided automatically if none is specified
 - it is **never** called explicitly (never called by the programmer)
 - it is **always** called implicitly (automatically) at specific points in the program

Destructors (cont.)

- Uses of destructors:
 - they can perform clean-up when an object is destroyed
 - e.g. if an object contains members that are dynamically allocated
 - they can release resources, if required
 - close any open files
 - release dynamically allocated memory
 - this will be our main usage of destructors in this course
 - ... more on this later...
 - whatever else is needed, it depends on the class
- **You must write the destructor code yourself!**
 - the compiler will not know what clean-up is required

Destructors (cont.)

- Destructors are **always** called *implicitly*:
 - when a locally declared object moves out of scope
 - at the end of the program, for global objects
 - when dynamically allocated memory for an object is deallocated
 - using the `delete` operator
 - ... more on this later...
 - they are usually called in reverse order of constructor calls
 - but not always!
- **coding example** <p5>

Ctor/Dtor Order of Invocation

- For global scope objects:
 - the constructor is called:
 - before the `main()` function begins
 - in order of declaration
 - the destructor is called:
 - after the `main()` function terminates
 - if `exit()` is called
 - in reverse order of construction

Ctor/Dtor Order of Invocation (cont.)

- For local scope objects:
 - the constructor is called:
 - every time the object is declared
 - for example, at the beginning of a function, or a loop, or any block
 - the destructor is called:
 - when the control flow exits the local scope
 - at the end of the block where the object is declared

Ctor/Dtor Order of Invocation (cont.)

- Abnormal program termination
 - if **exit()** is called
 - it causes the immediate termination of the program
 - the destructors for the global objects will execute
 - the destructors for the local objects will **not** execute
 - if **abort()** is called
 - this indicates a non-recoverable failure
 - it causes the immediate termination of the program
 - the destructors for none of the objects will execute

1.3.6 Object Duplication

- What is an *object declaration*?
 - it declares an object variable
 - it allocates the memory for the object
 - it calls the class constructor to initialize the object
 - example: `Student matilda;`

Object Duplication (cont.)

- What is an *object initialization*?
 - it declares an object variable and allocates the memory for it
 - it initializes the object with values *from an existing object*
 - it does **not** call the default constructor
 - an object initialization triggers a call to the copy constructor
 - more on this in the next section
 - example: `Student matilda = berth;`

Object Duplication (cont.)

- What is an *object assignment* operation?
 - it copies one existing object into another existing object
 - no new objects are allocated, so no constructors are called
 - example: `matilda = berth;`
 - the default behaviour is a *shallow copy*
 - if we want a *deep copy*, we have to write our own code for the assignment operator
 - ... we talk about overloaded operators in a later section ...

1.3.7 Copy Constructors

- Constructors that take one argument are special
 - *copy constructor*
 - it's a constructor that takes one parameter: a *reference* to an object of the same class
 - not an object passed by value
 - not a pointer to an object
 - it can be called explicitly
 - it can also be called implicitly in unexpected ways! 
 - *conversion constructor*
 - it's a constructor that takes one parameter, of any type other than a reference to an object of the same class

Copy Constructors (cont.)

- Characteristics of a *copy constructor*:
 - it's a member function of a class
 - it takes one parameter: a reference to an object of the same class
 - only one copy constructor can exist for each class
 - a copy constructor is provided automatically if none is specified
 - the provided copy constructor performs *member-wise assignment*
 - member-wise assignment is basically a **shallow** copy!
 - this is bad news for data members that are pointers
 - it may be called explicitly

Copy Constructors (cont.)

- Uses of copy constructors:
 - they are used to make a copy of an existing object
 - they initialize a *new object* from the values of an *existing object*

Copy Constructors (cont.)

- Copy constructors can be called explicitly:
 - on declaration, when given a parameter value
- Copy constructors can be called implicitly:
 - when an object is passed by value as parameter to any function
- 
- on **initialization**
 - do not confuse this with **assignment**!
- **coding example** <p6>

1.3.8 Conversion Constructors

- Characteristics of a *conversion constructor*:
 - it's a member function of a class
 - it takes one parameter of a *different data type*
 - a data type not of a reference to an object of the class type
 - multiple conversion constructors can be implemented
 - their parameter data types must be different
 - it may be called either explicitly or implicitly, depending on syntax

Conversion Constructors (cont.)

- Uses of conversion constructors:
 - they are used to initialize a new object based on an existing variable
 - a primitive data type or an object of a different class
 - not every class needs conversion constructors, it's very class-dependent
- Issues with conversion constructors:
 - simple syntax errors can accidentally call a conversion constructor
 - the **explicit** keyword disables implicit calls to a conversion constructor
 - it still allows explicit calls

Conversion Constructors (cont.)

- Conversion constructors can be called explicitly:
 - on declaration, when given a parameter value
 - example: `Book b1; Movie m1(b1);`
- Conversion constructors can be called implicitly:
 - when a function expects an object but is given another type
 - for example:
 - a function expects a `Movie` object as parameter, but is passed a `Book` instead
 - if a `Movie` constructor exists that takes a `Book` parameter, a new `Movie` is created from that `Book`
 - on **initialization**
- **coding example** <p7>